

TP07 : Bataille Navale

Table des matières

1) Problématique.....	1
2) Etapes	1
3) Conclusion.....	7

1) Problématique

Nous cherchons à créer un jeu de bataille navale en java, pour faire cela nous allons suivre les étapes du TP et les réaliser sur Eclipse.

2) Etapes

Etape 1 : Initialisation des tableaux et attribution des valeurs :

Nous souhaitons créer un tableau pour le joueur et tableau pour l'ordi, le tableau doit être d'un format 5x5 et chaque cellule du tableau doit être attribué la valeur 0. Pour faire cela nous allons créer deux variables : **tabjoueur** et **tabordi** qui seront composés de 5 lignes et 5 colonnes :

```
int[][] tabjoueur = new int[5][5];  
int[][] tabordi = new int[5][5];
```

Puis nous allons attribuer la valeur 0 à chaque cellule du tableau en parcourant chaque i (qui représente les lignes) et chaque y (qui représente les colonnes). On utilise une boucle dans une boucle pour que les valeurs de i et y puissent changer à chaque passage, ce qui permet d'accéder à toutes les cases du tableau :

```
for (int i = 0; i < 5; i++) {  
    for (int y = 0; y < 5; y++) {  
        tabjoueur[i][y] = 0;  
        tabordi[i][y] = 0;  
    }  
}
```

Etape 2 : Placement des pions du joueur :

2.1), 2.2), 2.3) et 2.4)

On cherche à demander à l'utilisateur à quel ligne et colonne il veut placer son pion, il faut faire en sorte que si l'utilisateur choisit une valeur plus petite que 0 ou plus grande que 4, alors il devra réécrire une nouvelle valeur jusqu'à que ça marche. Il faut aussi faire en sorte de vérifier si l'emplacement du pion est déjà pris ou pas (donc s'il est égal à 0 ou non). Pour faire tout ça nous allons créer trois variables, la première sera **ligne** qui sera égal à la valeur que le joueur a choisi pour les lignes et le deuxième sera **colonne** qui sera égal à la valeur que le joueur aura choisi pour les colonnes. La troisième sera un booléen nommé **positionValide** qui permettra de vérifier si le placement des pions peut se faire ou non, si le pion est en dehors des limites alors le booléen restera faux et de même si l'emplacement est déjà pris. Et dans le cas où **positionValide** est faux alors on lui redemandera en boucle de donner les valeurs pour **ligne** et **colonne**.

Déclaration des variables et importation du scanner :

```
import java.util.Scanner;

public class Bataillenavala {
    public static void main(String[] args) {
        int[][] tabjoueur = new int[5][5];
        Scanner scanner = new Scanner(System.in);

        int ligne;
        int colonne;
        boolean positionValide;
```

Demande de l'emplacement des pions du joueur :

```
for (int n = 1; n <= 5; n++) {
    System.out.println("Placement du pion n°" + n);
    positionValide = false;

    while (!positionValide) {
        System.out.print("Choisissez une ligne (0 à 4) : ");
        ligne = scanner.nextInt();

        System.out.print("Choisissez une colonne (0 à 4) : ");
        colonne = scanner.nextInt();

        if (ligne < 0 || ligne >= 5 || colonne < 0 || colonne >= 5) {
            System.out.println("Position hors du tableau. Réessayez.");
        } else if (tabjoueur[ligne][colonne] != 0) {
            System.out.println("Case déjà occupée. Réessayez.");
        } else {
            tabjoueur[ligne][colonne] = 1;
            System.out.println("Pion placé en [" + ligne + "][" + colonne + "]\n");
            positionValide = true;
        }
    }
}
```

Etape 3 : Placement des pions de l'ordinateur :

3.1),3.2),3.3 et 3.4)

Cette fois-ci nous allons faire la même chose sauf que les variables **ligne** et **colonne** vont être choisis aléatoirement grâce à **random**. Les conditions sont les mêmes que pour le joueur, il suffit donc simplement de faire la même chose mais avec le calcul aléatoire des valeurs (on force l'ordi à choisir entre 1 et 5 donc pas besoin de vérifier et on peut faire en sorte qu'il change la valeur d'une case seulement si elle est égale à 0) :

```
Random random = new Random();

for (int n = 1; n <= 5; n++) {
    positionValide = false;

    while (!positionValide) {
        ligne = random.nextInt(5);
        colonne = random.nextInt(5);

        if (tabordi[ligne][colonne] == 0) {
            tabordi[ligne][colonne] = 1;
            positionValide = true;
        }
    }
}
```

Etape 4 : Création de la procédure *affichagetab* :

4.1),4.2), 4.3,4.4) et 4.5)

Maintenant nous allons créer un fichier procédure qui permettra de créer un visuel du tableau en remplaçant les 1 par des o et les 0 par des ~. La procédure aura en paramètre un tableau et une variable nommée **nbcases** qui déterminera le nombre de cases (dans notre cas 5). Puis lorsque c'est fait on fait appel à la procédure dans le code principal (j'ai eu beaucoup de mal pour les espaces) :

```
public static void affichagetabJoueur(int[] [] tableau, int nbcase) {  
    System.out.print("    ");  
    for (int y = 1; y <= nbcase; y++) {  
        System.out.print(y + "    ");  
    }  
    System.out.println();  
  
    for (int i = 0; i < nbcase; i++) {  
        System.out.print((i + 1) + "    ");  
        for (int y = 0; y < nbcase; y++) {  
            if (tableau[i][y] == 1) {  
                System.out.print("o    ");  
            } else {  
                System.out.print("~    ");  
            }  
        }  
        System.out.println();  
    }  
}
```

```
affichagetabJoueur(tabjoueur, 5);
```

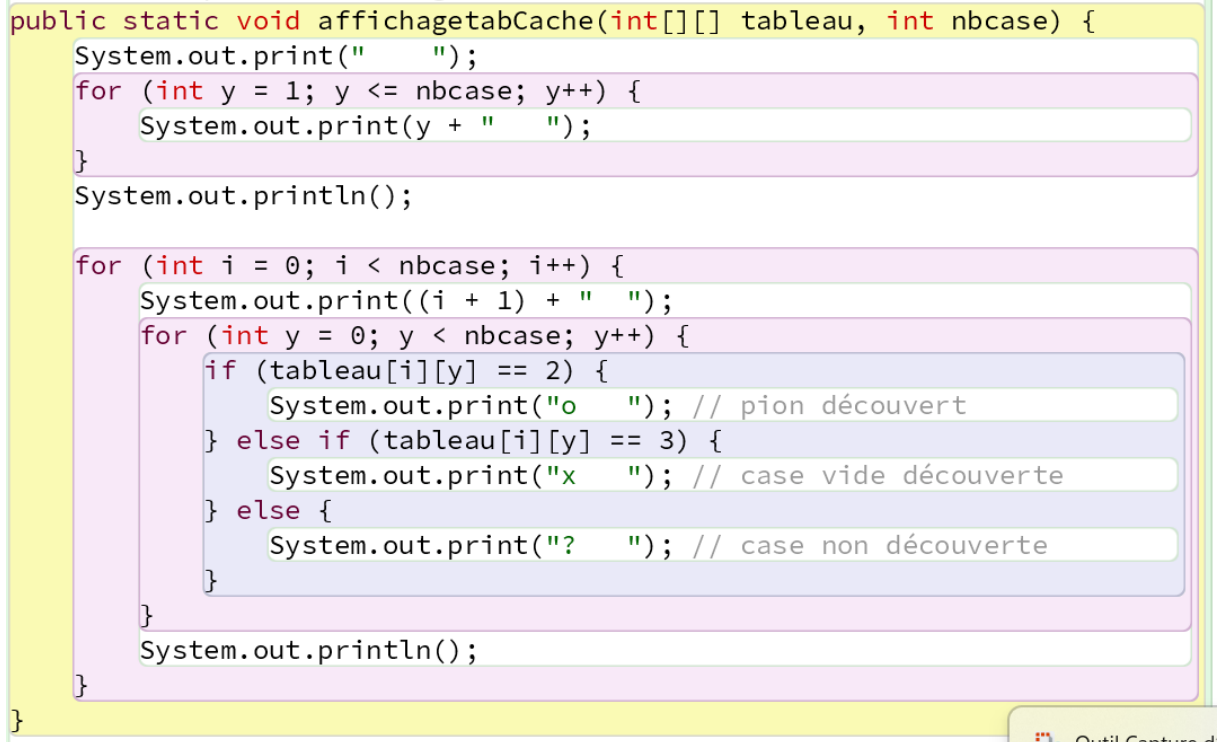
Résultat :

	1	2	3	4	5
1	~	~	~	~	~
2	~	~	o	o	o
3	~	~	~	o	~
4	~	~	~	~	o
5	~	~	~	~	~

Etape 5 : Création de la procédure *affichagetabCache* :

A présent nous souhaitons faire en sorte que les cases qui ne sont pas touchées affiche un ? comme ça le joueur ne sait pas où se trouve les pions adverses. Pour cela on va rajouter les états 2 et 3 qui indiqueront dans l'ordre respectif pion découvert et case découverte sans pion :

```
public static void affichagetabCache(int[][] tableau, int nbcase) {  
    System.out.print("    ");  
    for (int y = 1; y <= nbcase; y++) {  
        System.out.print(y + "    ");  
    }  
    System.out.println();  
  
    for (int i = 0; i < nbcase; i++) {  
        System.out.print((i + 1) + " ");  
        for (int y = 0; y < nbcase; y++) {  
            if (tableau[i][y] == 2) {  
                System.out.print("o "); // pion découvert  
            } else if (tableau[i][y] == 3) {  
                System.out.print("x "); // case vide découverte  
            } else {  
                System.out.print("? "); // case non découverte  
            }  
        }  
        System.out.println();  
    }  
}
```



```
affichagetabCache(tabordi, nbcase);
```

Etape 6 : Tour du joueur :

Pour faire le tour du joueur j'ai plutôt opté à faire une fonction à part pour le tir du joueur. La fonction retourne 1 si le joueur touche un pion, sinon elle retourne 0. Lorsqu'un pion est touché la variable **nbPionTrouvéJoueur** augmente de 1.

```
public static int tirJoueur(int[][] tabordi, int nbcase, Scanner scanner) throws InterruptedException {
    System.out.println("\nÀ vous de jouer !");
    System.out.print("Choisissez une ligne (1 à " + nbcase + ") : ");
    int ligne = scanner.nextInt() - 1;
    System.out.print("Choisissez une colonne (1 à " + nbcase + ") : ");
    int colonne = scanner.nextInt() - 1;

    System.out.println("Tir en cours...");
    Thread.sleep(2000);

    if (tabordi[ligne][colonne] == 0) {
        System.out.println("Raté !");
        tabordi[ligne][colonne] = 3;
    } else if (tabordi[ligne][colonne] == 1) {
        System.out.println("Touché !");
        tabordi[ligne][colonne] = 2;
        afficheTabCache(tabordi, nbcase);
        return 1;
    } else {
        System.out.println("Tir à blanc (déjà tiré ici !)");
    }

    afficheTabCache(tabordi, nbcase);
    return 0;
}
```

```
int nbPionTrouveJoueur = 0;
nbPionTrouveJoueur += tirJoueur(tabordi, 5, scanner);
```

Etape 7 : Tour de l'ordinateur :

Même chose mais avec l'ordi, on incrémente **nbPionTrouvéOrdi** cette fois-ci :

```
public static int tirOrdi(int[][] tabjoueur, int nbcase, Random random) throws InterruptedException {
    System.out.println("\nTour de l'ordinateur !");
    int ligne;
    int colonne;
    boolean positionValide = false;

    do {
        ligne = random.nextInt(nbcase);
        colonne = random.nextInt(nbcase);
        if (tabjoueur[ligne][colonne] == 0 || tabjoueur[ligne][colonne] == 1) {
            positionValide = true;
        }
    } while (!positionValide);

    System.out.println("L'ordinateur tire sur [" + (ligne + 1) + "][" + (colonne + 1) + "]...");
    Thread.sleep(2000);

    if (tabjoueur[ligne][colonne] == 0) {
        System.out.println("L'ordinateur a raté !");
        tabjoueur[ligne][colonne] = 3;
    } else if (tabjoueur[ligne][colonne] == 1) {
        System.out.println("L'ordinateur a touché un de vos pions !");
        tabjoueur[ligne][colonne] = 2;
        afficheTabJoueur(tabjoueur, nbcase);
        return 1;
    }

    afficheTabJoueur(tabjoueur, nbcase);
    return 0;
}
```

```
nbPionTrouveOrdi += tirOrdi(tabjoueur, 5, random);
```

Etape 8 : Fin de partie :

Enfin, nous souhaitons faire en sorte d'arrêter le jeu lorsqu'un joueur atteint un score de 5. Ensuite nous afficherons le vainqueur et finalement nous allons demander au joueur si il veut rejouer :

```
if (nbPionTrouveJoueur == 5) {  
    System.out.println("\n Vous avez gagné ! Tous les pions ennemis sont touchés !");  
    break;  
}  
  
if (nbPionTrouveOrdi == 5) {  
    System.out.println("\n L'ordinateur a gagné ! Tous vos pions ont été détruits !");  
    break;  
}  
  
System.out.print("\nVoulez-vous rejouer ? (o/n) : ");  
char reponse = scanner.next().toLowerCase().charAt(0);  
if (reponse == 'o') {  
    System.out.println("\nNouvelle partie !");  
    main(args); // relance le jeu  
} else {  
    System.out.println("Fin du jeu. Merci d'avoir joué !");  
}
```

3) Conclusion

En conclusion, j'ai pu réussir à créer un jeu de bataille navale en suivant les étapes du TP et j'ai pu mettre en pratique le système de fonctions et de procédures pour m'aider à créer ce jeu.